

목차

1. 해시
2. 대칭키/비대칭키
3. https

요약

 암호 알고리즘의 세 가지 큰 범주				
구분	주요 알고리즘 예시	특징	키 사용	대표 OpenSSL 명령 예
① 해시(Hash)	MD5, SHA-1, SHA-256, SHA3, BLAKE2 등	입력 데이터를 고정 길이 요약값으로 변환	❌ (키 없음)	<code>openssl sha256 file.txt</code>
② 대칭키 (Symmetric)	AES, DES, 3DES, ChaCha20, Blowfish 등	암호화·복호화 시 같은 키 사용	✅ (1개의 비밀키)	<code>openssl enc -aes-256-cbc -in a.txt -out a.enc -pass pass:1234</code>
③ 비대칭키 (Asymmetric)	RSA, ECC(ECDSA, Ed25519), DSA 등	공개키와 비밀키 쌍 사용	✅ (공개키·개인키 2개)	<code>openssl rsautl -encrypt -inkey pub.pem -pubin -in a.txt -out a.enc</code>

단방향

양방향

양방향

- ◆ 해시: “비밀번호를 못 보게 요약하는 함수” (키 없음)
- ◆ 대칭키: “같은 열쇠로 잠그고 여는 자물쇠”
- ◆ 비대칭키: “공개키로 잠그고, 개인키로 여는 자물쇠”

1. 해시

비밀번호 저장

1. 해시

해시 – 데이터 무결성

✓ 문자열 해시

```
C:\Users\user\workspaces\coding_assistant> echo password | openssl sha256  
SHA2-256(stdin)= 7dd7367c6b2a075cf967e12b81eabec7f2358fcab4dc922e38e65109a7b7f413
```

✓ 파일 해시

```
C:\Users\user\workspaces\coding_assistant> openssl sha256 file.txt  
SHA2-256(file.txt)= b94d27b9934d3e08a52e52d7da7dabfac484efe37a5380ee9088f7ace2efcde9
```

```
file.txt  
1 hello world
```

1. 해시

해시 알고리즘

알고리즘	출력 길이	보안성(충돌)	속도(대략)	호환성/표준	OpenSSL 명령 예시	권장 용도
MD5	128bit (32hex)	깨짐(충돌 있음) ❌	매우 빠름	매우 널리 호환	<code>openssl md5 file.txt</code> 또는 <code>openssl dgst -md5 file.txt</code>	교육/비보안 무결성 체크만 (권장X)
SHA-1	160bit (40hex)	깨짐(충돌 있음) ❌	빠름	널리 호환(과거)	<code>openssl sha1 file.txt</code>	과거 호환 필요 시만(권장X)
SHA-224	224bit	안전(권장 ↓)	보통	표준(SHA-2)	<code>openssl sha224 file.txt</code>	특별 필요 없으면 SHA-256 권장
SHA-256	256bit (64hex)	안전 ✅	보통	표준/가장 보편	<code>openssl sha256 file.txt</code> 또는 <code>openssl dgst -sha256 file.txt</code>	일반적 파일/배포 무결성, 서 명 전 해시
SHA-384	384bit	안전 ✅	보통~조금 느림	표준(SHA-2)	<code>openssl sha384 file.txt</code>	고보안 환경(서명체계 등)
SHA-512	512bit (128hex)	안전 ✅	64-bit CPU에서 빠름	표준(SHA-2)	<code>openssl sha512 file.txt</code>	대용량 파일, 64-bit 환경에서 성능 우선
SHA3-256	256bit	안전 ✅	SHA-2보다 느림	표준(SHA-3)	<code>openssl dgst -sha3-256 file.txt</code>	SHA-3 필요 정책/연구
BLAKE2s-256	256bit	안전 ✅	빠름	표준(RFC 7693)	<code>openssl dgst -blake2s256 file.txt</code>	빠른 무결성 체크(지원 시)
BLAKE2b-512	512bit	안전 ✅	빠름(64-bit 우수)	표준(RFC 7693)	<code>openssl dgst -blake2b512 file.txt</code>	고속+강력 해시(지원 시)
BLAKE3-256	256bit	안전 ✅	매우 빠름	비표준(별도 lib)	(기본 OpenSSL 미지원)	자체 툴/라이브러리 사용 시

1. 해시

비밀번호 저장 방식

유저	비밀번호	Salt	Hash Value (SHA-256)
User1	password1	<u>+ fw32</u>	1034eb45dc9270975a01c4388762a553 cb956efe7f9e87e5f437507af475c755
User2	holidaygood	<u>+ abc1</u>	9b3522fa70cc3b9a029059d8264e0da7 859376a58d360ad7f91d548759a4707c
User3	moreholiday	<u>+ 54k3</u>	d44f21cc93d6bcd6d888b42c08e8f1ce1 a9bdc8c8a6bcb2c5841326f9fde9c2
User4	itsgoingover	<u>+ gdl3</u>	97f33442cd68fa593042c6ee070165af5 41322b671462481a1b64c741b5eb7db

1. 해시

비밀번호 해킹

✔ 무차별 대입 공격(Brute-Force Attack)

무차별 대입 공격은 가장 직접적이고 단순한 해킹 방법입니다.

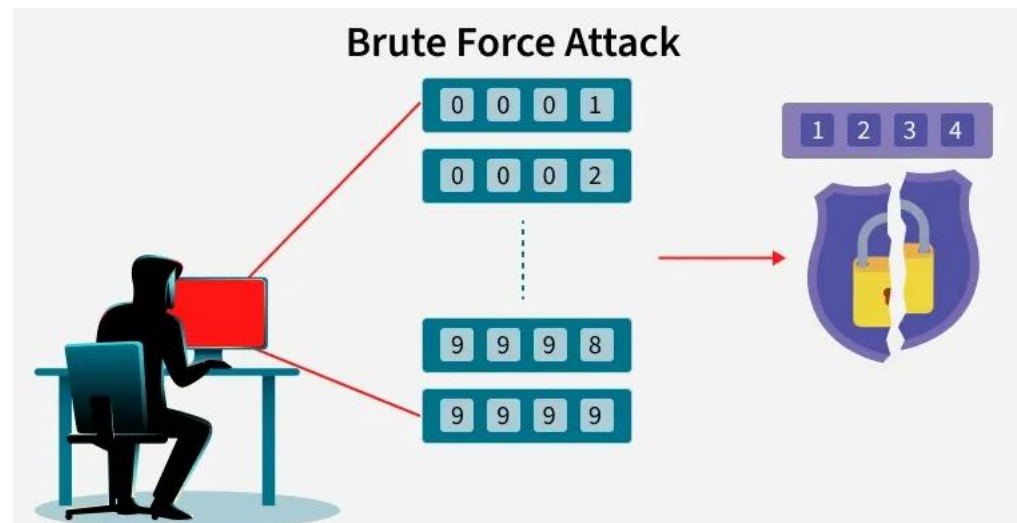
작동 방식

해커는 특정 비밀번호에 대한 모든 가능한 문자열 조합을 시도하여, 각각을 해시 함수에 적용해 공격 대상의 저장된 해시 값과 비교합니다.

- **예시:** 비밀번호가 4자리 숫자라면, 해커는 0000, 0001, 0002, ..., 9999까지 모든 조합을 차례로 시도합니다.
- **시간 소요:** 비밀번호의 길이가 길어지거나, 사용 가능한 문자 종류(대문자, 소문자, 숫자, 특수문자)가 늘어날수록 조합의 수가 기하급수적으로 증가하여 공격에 필요한 시간이 급격히 늘어납니다.

대응 방법

- **비밀번호 길이 및 복잡성 강화:** 비밀번호 길이를 12자 이상, 문자 종류를 다양하게 설정하도록 강제합니다.
- **계정 잠금 (Rate Limiting):** 일정 횟수 이상 로그인 실패 시 계정을 잠가서 공격 속도를 제한합니다.
- **Salt와 Iteration:** 키 도출 함수(KDF)를 사용하여 해시를 생성할 때 **반복 횟수(Iteration)**를 늘려 해시 생성 자체를 느리게 만듭니다.



1. 해시

비밀번호 해킹

✓ 레인보우 테이블 공격(Rainbow Table Attack)

레인보우 테이블 공격은 무차별 대입 공격의 효율을 높이기 위해 시간과 저장 공간을 교환하는 방식입니다.

작동 방식

이 공격은 해커가 비밀번호를 찾기 위해 **미리 계산해 둔 해시 값 목록(레인보우 테이블)**을 사용하는 것이 핵심입니다.

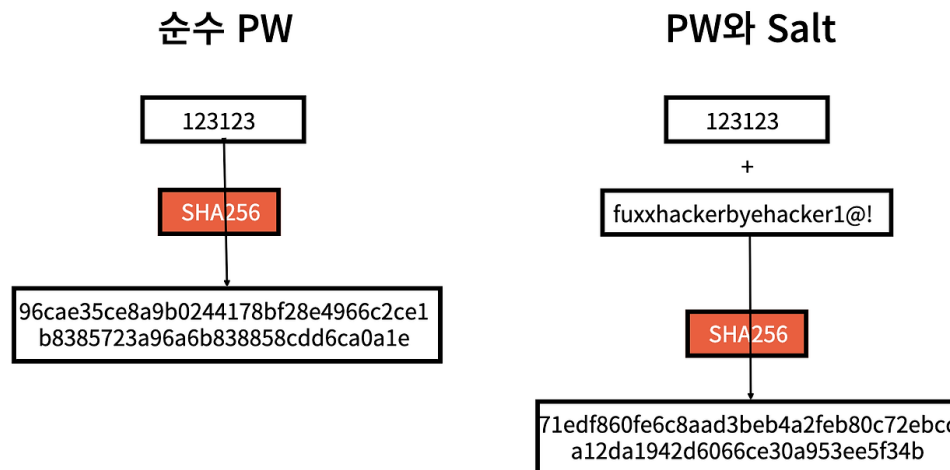
1. **사전 계산 (Pre-computation):** 해커는 해킹 전에 수많은 일반적인 비밀번호와 그 해시 값을 미리 계산하여 테이블로 만듭니다.
2. **역추적 시도:** 해커는 공격 대상 시스템에서 탈취한 **해시 값**을 준비된 레인보우 테이블에서 곧바로 검색합니다.
3. **성공:** 만약 일치하는 해시 값이 테이블에 있다면, 그 해시에 해당하는 **원본 비밀번호**를 즉시 얻게 됩니다.

이 방식은 수많은 비밀번호를 재계산하는 무차별 대입 공격보다 훨씬 빠르게 비밀번호를 찾을 수 있습니다.

대응 방법

- **솔트(Salt) 사용:** 레인보우 테이블 공격을 막는 가장 효과적인 방법입니다.
 - 솔트는 사용자의 비밀번호에 무작위로 생성된 데이터를 추가한 후 해시를 계산하는 기법입니다.
 - 솔트를 사용하면 같은 비밀번호를 가진 사용자라도 각자 **다른 해시 값**을 갖게 됩니다.
 - 레인보우 테이블은 특정 **해시 함수에 솔트가 없는 경우**를 가정하고 만들어지므로, 솔트가 적용된 해시 값에는 테이블을 사용할 수 없게 됩니다. 해커는 사용자마다 레인보우 테이블을 새로 만들어야 하므로 공격이 비실용적이 됩니다.
- **느린 해시 함수 사용:** bcrypt, scrypt, Argon2와 같은 **키 도출 함수(KDF)**를 사용하여 해시 계산 자체를 느리게 만듭니다.

비밀번호	해시 값
password	5BAA61E4C9B93F3F0682250B6CF8331B7EE68FD8
Password	8BE3C943B1609FFFBF51AAD666D0A04ADF83C9D
password123	CBFDAC6008F9CAB4083784CBD1874F76618D2A97
123456	7C4A8D09CA3762AF61E59520943DC26494F8941B
qwerty	B1B3773A05C0ED0176787A4F1574FF0075F7521E
qwerty1234	153FA238CEC90E5A24B85A79109F91EBE68CA481
1q2w3e4r	48EFC4851E15940AF5D477D3C0CE99211A70A3BE
abcd1234	7CE0359F12857F2A90C7DE465F40A95F01CB5DA9
asdasd	85136C79CBF9FE36BB9D05D0639C70C265C18D37
qweqwe	F4542DB9BA30F7958AE42C113DD87AD21FB2EDDB
123123	601F1889667EFAEBB33B8C12572835DA3F027F78



1. 해시

해시 - 비밀번호

1

```
C:\Users\user\workspaces\coding_assistant> openssl passwd -5 -salt MySalt
```

```
Password: password
```

```
$5$MySalt$tNQL2xei6N2UXO.s0868vNZNQUNN40tt/tkIl.uVI6A
```

```
C:\Users\user\workspaces\coding_assistant> openssl passwd -5 -salt MySalt
```

```
Password: password
```

```
$5$MySalt$tNQL2xei6N2UXO.s0868vNZNQUNN40tt/tkIl.uVI6A
```

```
C:\Users\user\workspaces\coding_assistant> openssl passwd -5
```

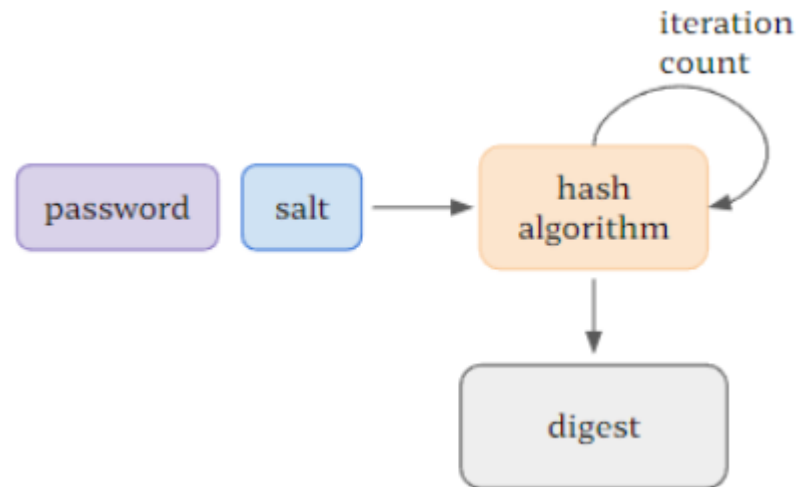
```
Password: password
```

```
Verifying - Password:
```

```
$5$PIIPTtI7E3h1zh03$4ki/PtMljHjW3t68SkCWpzvu9/DSendO/m/VHAHefd3
```

출력값 형식: 알고리즘\$솔트\$해시값

옵션	알고리즘 식별자	알고리즘	보안 수준	설명
(기본값)	(없음)	DES-crypt	매우 낮음	기본값. 솔트 최대 2자, 비밀번호 최대 8자 제한. 매우 취약하여 사용하지 않아야 합니다.
-1	\$1\$	MD5-crypt	낮음	MD5 기반의 비밀번호 해시 알고리즘. MD5 자체가 취약하며 반복 횟수가 낮아 현재는 사용이 권장되지 않습니다. (솔트 최대 8자)
-5	\$5\$	SHA-256-crypt	높음	권장. SHA-256 기반. 솔트와 반복 횟수(약 5,000회)를 사용하여 보안성이 높습니다.
-6	\$6\$	SHA-512-crypt	가장 높음	가장 권장. SHA-512 기반. SHA-256보다 더 강력하고 더 많은 반복 횟수를 사용합니다.



2. 대칭키/비대칭키

암호화

2. 대칭키/비대칭키

암호 알고리즘

✓ 대칭키



✓ 비대칭키



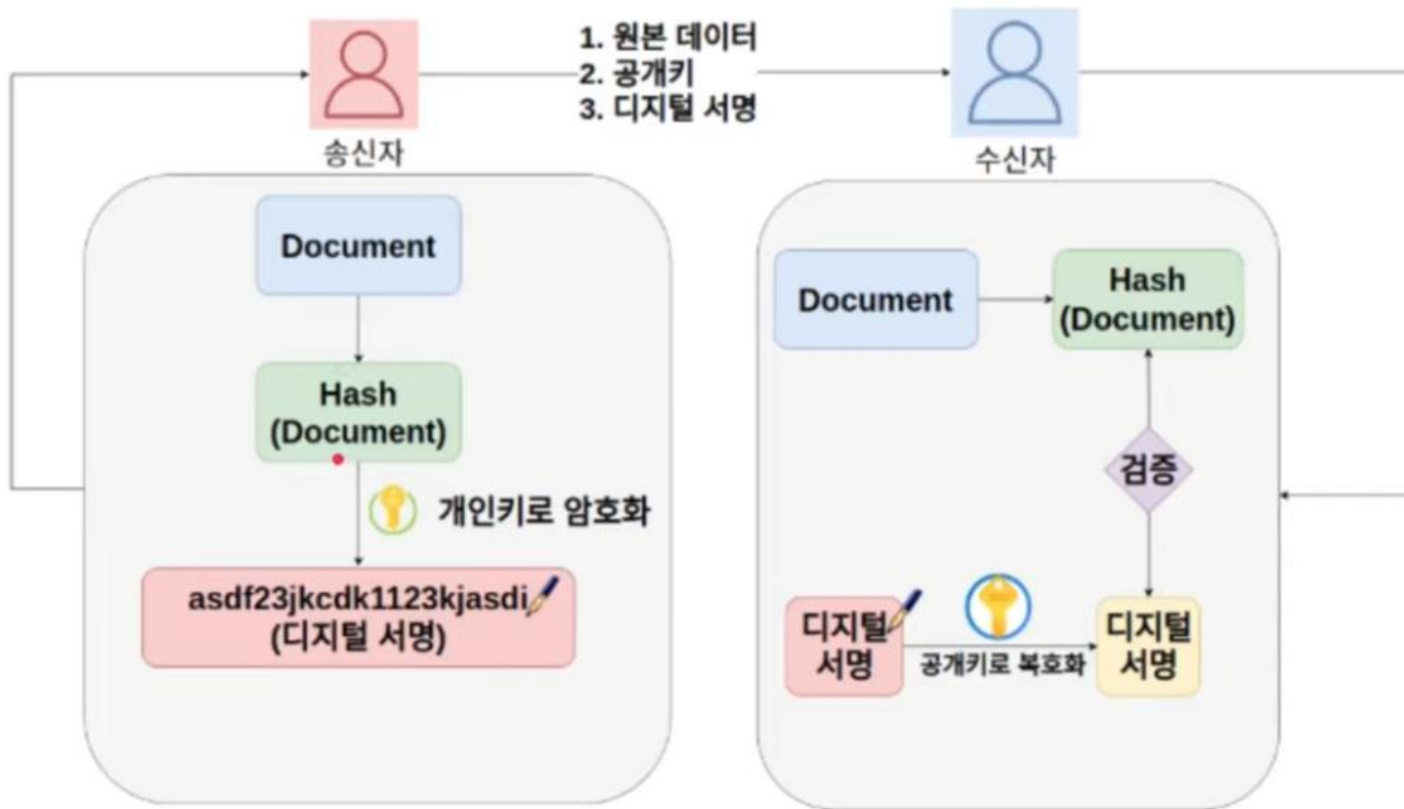
2. 대칭키/비대칭키

비교

구분	대칭키 암호화 (Symmetric Encryption)	비대칭키 암호화 (Asymmetric Encryption)
키 구조	암호화와 복호화에 같은 키 사용	**공개키(public)**로 암호화, **개인키(private)**로 복호화
대표 알고리즘	AES, DES, 3DES, ChaCha20, Blowfish 등	RSA, ECC, DSA, ElGamal 등
키 길이	짧음 (128~256bit)	김 (1024~4096bit 이상)
속도	매우 빠름 ⚡	느림 🐢
보안성	키 유출 시 복호화 가능 (키 공유 문제)	공개키는 배포 가능, 개인키만 보관
용도	데이터 암호화 (파일, 통신 내용 등)	키 교환, 디지털 서명, 인증서
OpenSSL 예시	<pre>openssl enc -aes-256-cbc -in a.txt -out a.enc -pass pass:1234</pre>	<pre>openssl rsautl -encrypt -inkey pub.pem -pubin -in a.txt -out a.enc</pre>
장점	빠르고 구현 쉬움	안전한 키 분배, 신뢰성 있는 서명 가능
단점	키를 안전하게 공유하기 어려움	느리고 계산량 많음

2. 대칭키/비대칭키

디지털 서명



- 디지털 서명: 암호화 된 일련의 문자열
- 개인키 암호화 방식 사용
- 송신자의 신원을 증명
- 데이터 무결성 검증
- 데이터 진위 검증에 사용
- 부인방지

2. 대칭키/비대칭키

대칭키 실습

✓ 대칭키

```
plain.txt  
1 hello world
```

```
C:\Users\user\workspaces\coding_assistant> openssl enc -aes-256-cbc -in plain.txt -out cipher.enc -pass pass:mypassword  
C:\Users\user\workspaces\coding_assistant> openssl enc -d -aes-256-cbc -in cipher.enc -out result1.txt -pass pass:mypassword
```

알고리즘 / 입력파일 / 출력파일 / 키

알고리즘	키 길이	블록 크기	명령어 예시
AES-128-CBC	16바이트(128bit)	16B	-aes-128-cbc
AES-192-CBC	24바이트(192bit)	16B	-aes-192-cbc
AES-256-CBC	32바이트(256bit)	16B	-aes-256-cbc
ChaCha20	32바이트(256bit)	12B (Nonce)	-chacha20

2. 대칭키/비대칭키

비대칭키 실습

✓ 공개키 암호화 / 개인키 복호화

```
plain.txt
1 hello world
```

개인키 생성

```
C:\Users\user\workspaces\coding_assistant> openssl genrsa -out private.pem 2048
```

공개키 추출

```
C:\Users\user\workspaces\coding_assistant> openssl rsa -in private.pem -pubout -out public.pem
```

공개키로 암호화

```
C:\Users\user\workspaces\coding_assistant> openssl rsautl -encrypt -inkey public.pem -pubin -in plain.txt -out msg.enc
```

개인키로 복호화

```
C:\Users\user\workspaces\coding_assistant> openssl rsautl -decrypt -inkey private.pem -in msg.enc -out result2.txt
```

2. 대칭키/비대칭키

비대칭키 실습

✓ 개인키 서명 / 공개키 검증

```
plain.txt
1 hello world
```

개인키 서명

```
C:\Users\user\workspaces\coding_assistant> openssl dgst -sha256 -sign private.pem -out signature.bin plain.txt
```

공개키 검증

```
C:\Users\user\workspaces\coding_assistant> openssl dgst -sha256 -verify public.pem -signature signature.bin plain.txt
```

Verified OK

✓ 대표 알고리즘

구분	알고리즘	수학적 기반	키 길이(비트)	속도	특징 / 사용처
RSA	RSA (Rivest-Shamir-Adleman)	큰 수 소인수분해	1024~4096	중간	가장 전통적이고 널리 사용. 서명·암호화 둘 다 가능.
DSA	Digital Signature Algorithm	이산 로그 문제 (mod p)	1024~3072	빠름(서명만)	전자서명 전용. 미국 NIST 표준.

3. https

http + secure

3. HTTPS

HTTP vs HTTPS



HTTP(HYPERTEXT TRANSFER PROTOCOL)

웹 상에서 클라이언트와 서버가 서로 정보를 주고받을 수 있도록 하는 규약

HTTPS(HyperText Transfer Protocol over Secure Socket Layer)

HTTP를 보완하는 수단

HTTPS는 HTTP의 일반 텍스트(text)에 SSL이나 TLS프로토콜을 씌워 통해 데이터를 암호화하는 기법
로그인이나 결제화면에서 주로 사용

관련 용어

1. 암호화: 어떤 정보를 암호화된 정보로 변환
2. 복호화: 암호화된 정보를 다시 원래 정보 변환
3. 키: 암호화, 복호화할 때 쓰는 비밀번호
4. HTTPS는 대칭키, 공개키 방식 사용

3. HTTPS

HTTPS 이해하기

대칭키

- 동일한 키로 암호화 복호화 수행

공개키(비대칭키)

- A키로 암호화를 하면 B키로 복호화 할 수 있고, B키로 암호화하면 A키로 복호화 할 수 있는 방식

인증서의 역할

- 클라이언트가 접속한 서버가 신뢰 할 수 있는 서버임을 보장한다.
- SSL 통신에 사용할 공개키를 클라이언트에게 제공한다.

CA(Certificate authority, 인증기관)

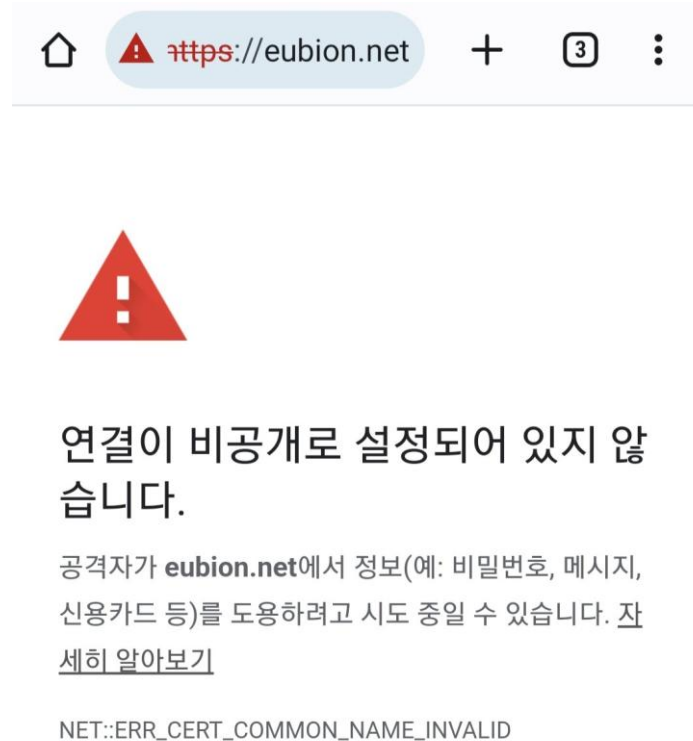
인증서의 역할은 클라이언트가 접속한 서버가 클라이언트가 의도한 서버가 맞는지를 보장하는 역할
이 역할을 하는 민간기업들이 있는데 이런 기업들을 CA(Certificate authority)라고 부름
CA는 아무 기업이나 할 수 있는 것이 아니고 신뢰성이 엄격하게 공인된 기업들 만이 참여 가능
CA는 브라우저가 알고 있음

3. HTTPS

주요 경고 메시지



http로 접속한 경우



인증서가 유효하지 않은 경우

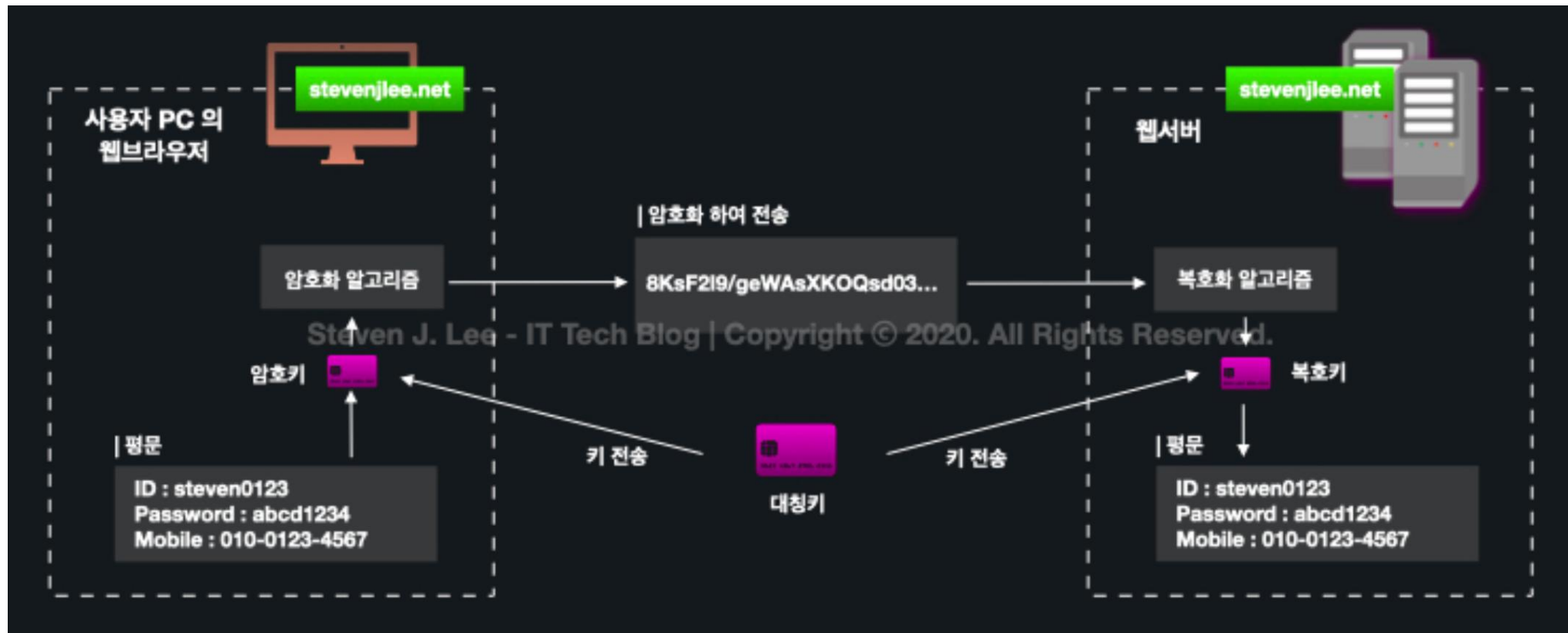
3. HTTPS

평문 전송



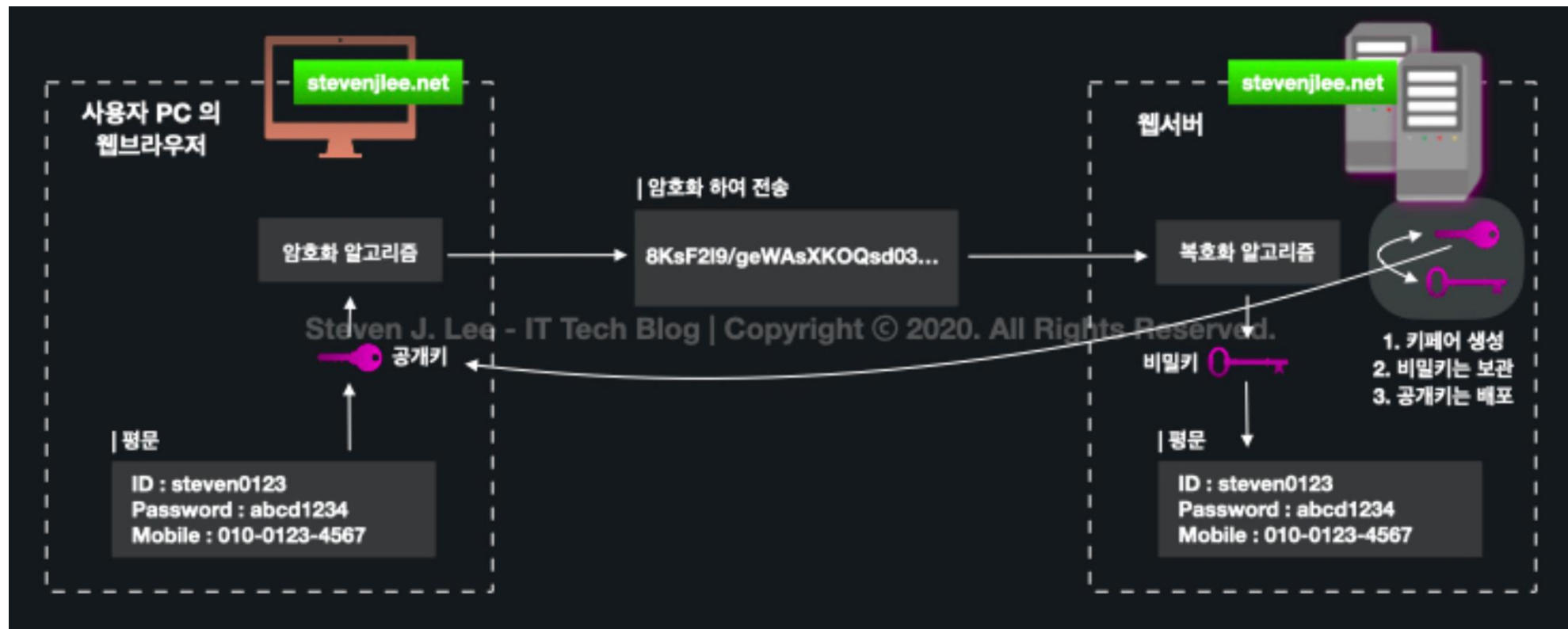
3. HTTPS

대칭키



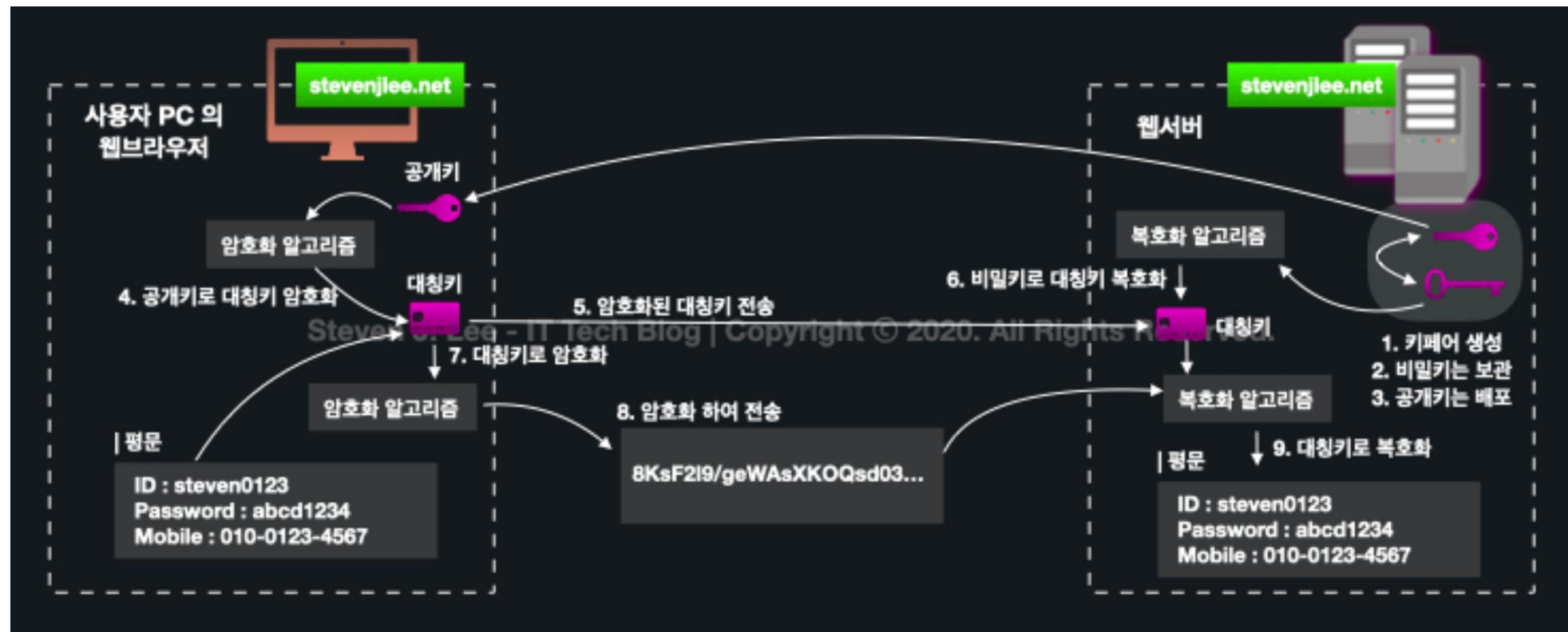
3. HTTPS

비대칭키(공개키, 비밀키)



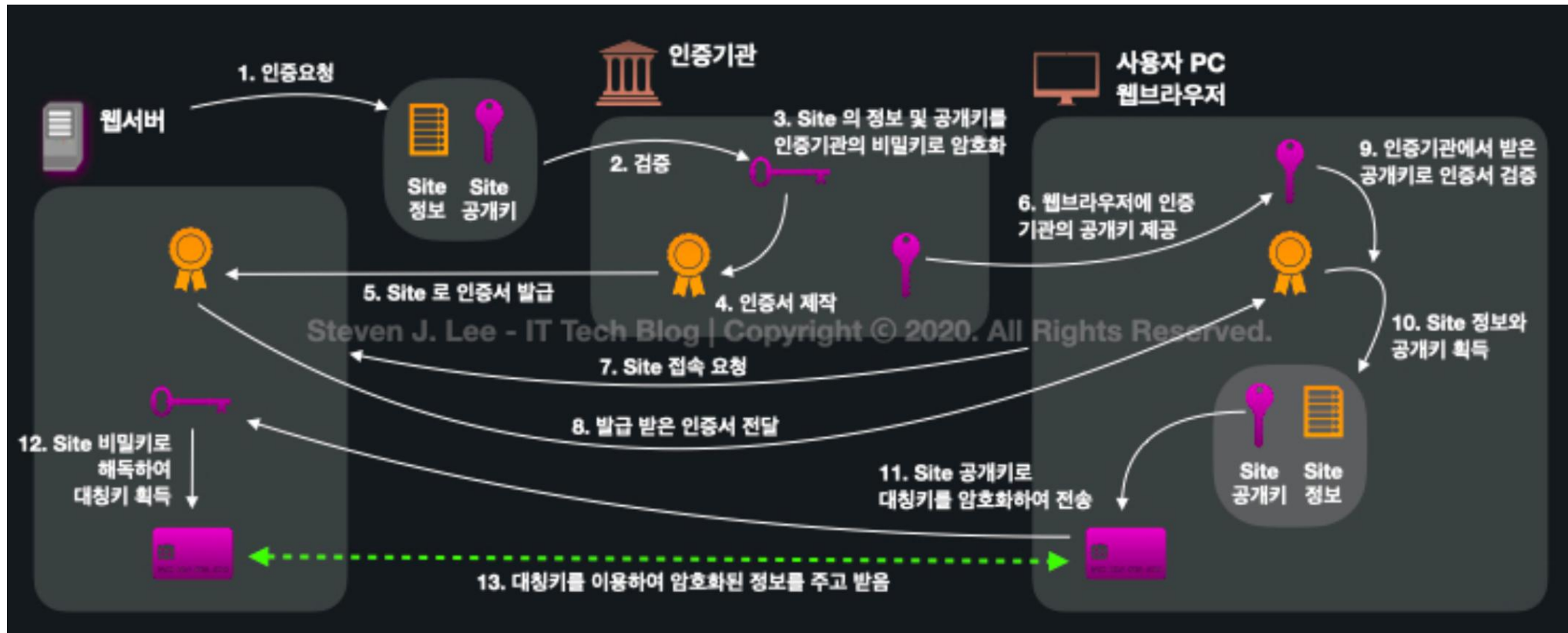
3. HTTPS

하이브리드



2. HTTPS

전체과정



3. HTTPS

인증서 실습

✓ 인증서 생성(생략)

- mkcert 사용(실습에서는 인증서 생성 생략)
- 웹 개발자들이 로컬(내 컴퓨터) 환경에서 HTTPS 테스트를 쉽게 하기 위해 만든 인증서 자동 생성 도구

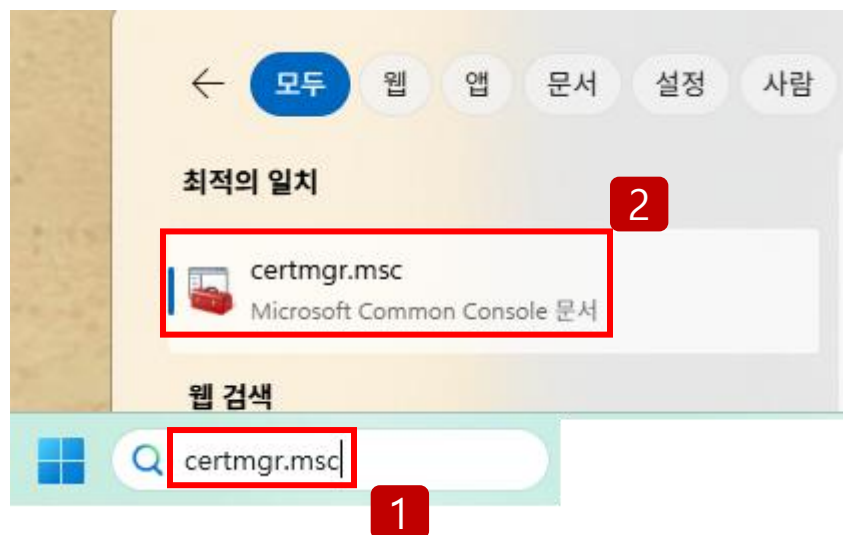
✓ 필요 파일

- rootCA.pem: 인증기관 인증서
- certificate(public).pem: 서버 인증서(서버 공인키 포함)
- private-key.pem: 서버 비밀키

3. HTTPS

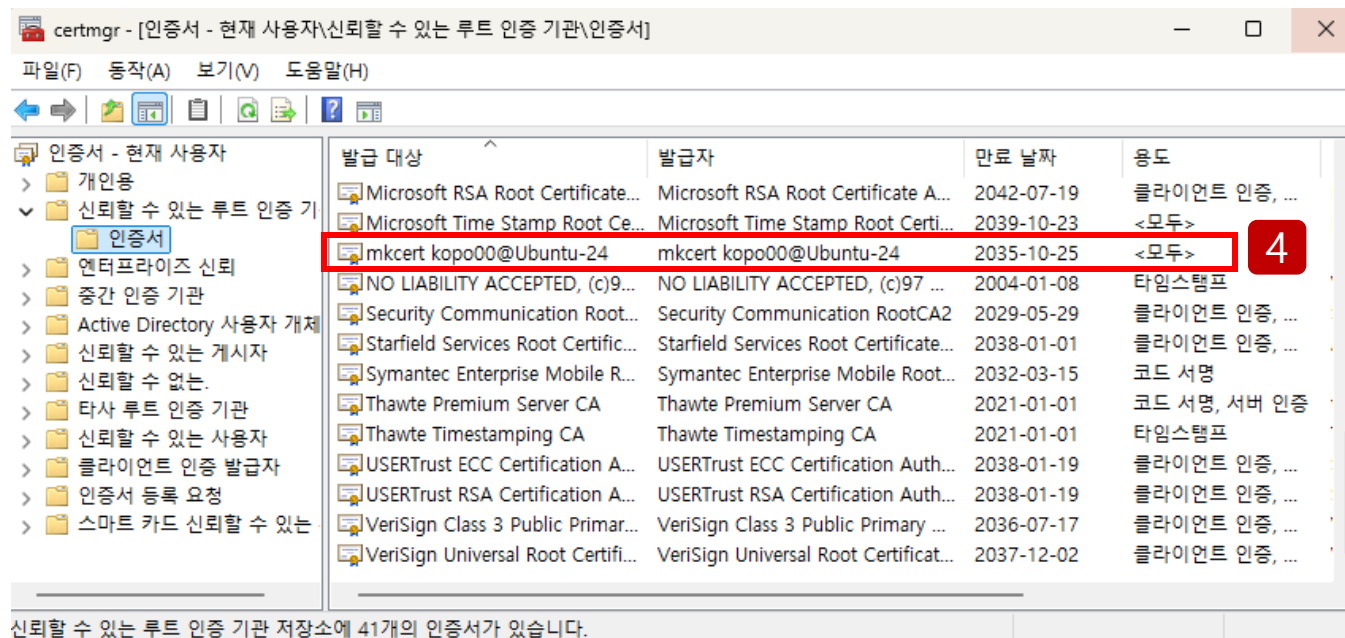
인증서 실습

✓ 신뢰할 수 있는 루트 인증 기관 등록



3

- "신뢰할 수 있는 루트 인증 기관" → "인증서" → 마우스 우클릭 "모든 작업" → "가져오기" → rootCA.pem 선택 → 마침



3. HTTPS

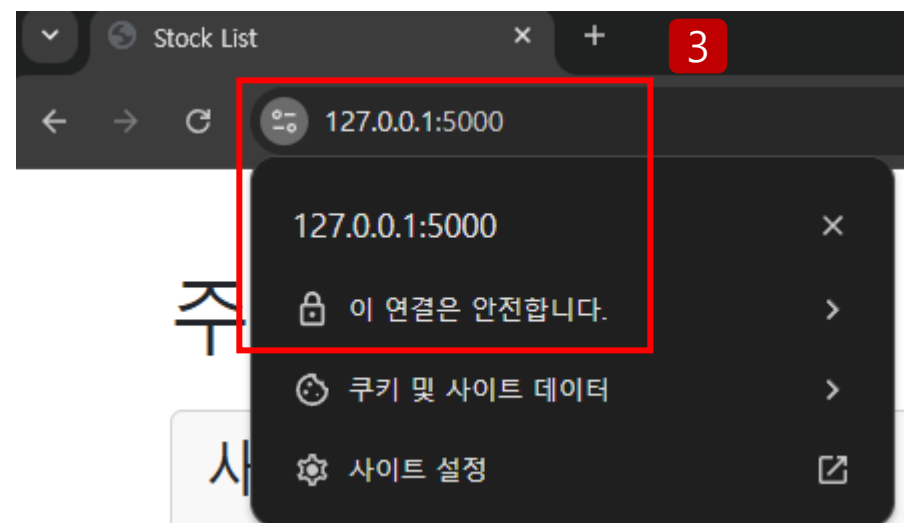
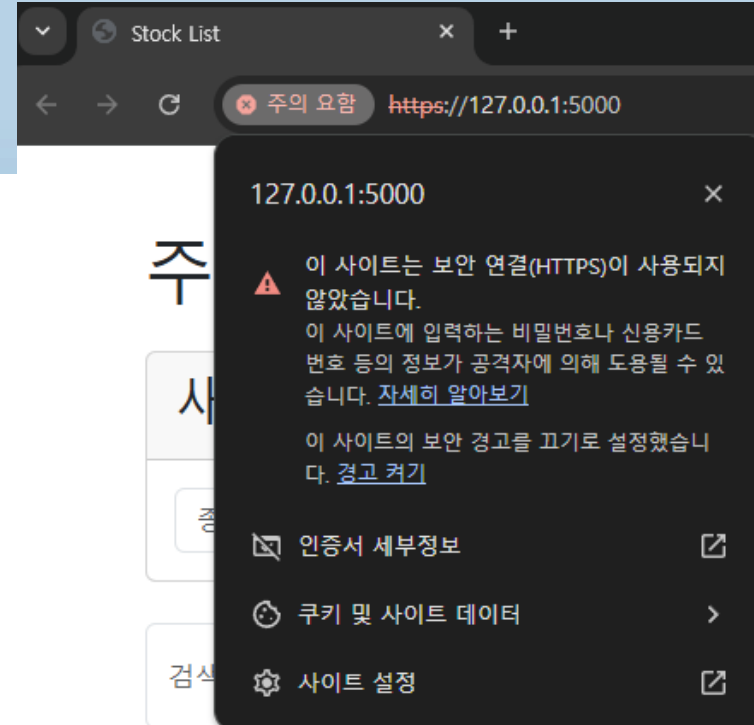
인증서 실습

✓ 서버 실행시 인증서, 비밀키 등록

1 flask 서버 실행시 certificate(public).pem, private-key.pem을 사용하여 서버를 실행하는 코드로 변경해줘.

```
if __name__ == '__main__':  
    app.run(debug=True, ssl_context=('certificate(public).pem', 'private-key.pem'))
```

*경고가 계속 뜨는 경우 시크릿 창으로 실행(캐시 때문)



3. HTTPS

인증서 실습

rootCA.pem(CA 인증서=인증기관의 공개키 + 식별정보)

인증서
certificate(public).pem

비밀키
private-key.pem

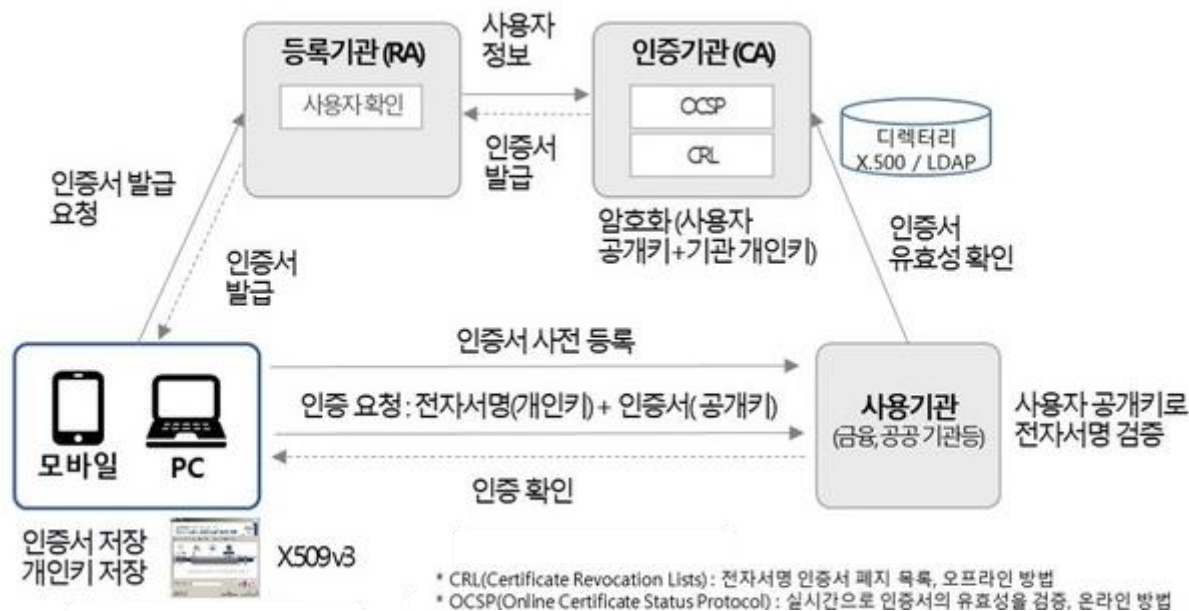


4. 공인인증서

x509v3

4. 공인인증서

작동원리



- X.509v3: 디지털 인증서 형식, 오늘날 인터넷 보안에서 가장 널리 쓰이는 표준
- 내용: "이 공개키는 누구의 것인가?"를 신뢰할 수 있게 보증하는 문서
- 구조

```
Certificate
├─ Version (v3)
├─ Serial Number
├─ Signature Algorithm
├─ Issuer (발급자)
├─ Validity (유효기간)
├─ Subject (소유자)
├─ Subject Public Key Info (공개키)
├─ Extensions (확장 필드)
└─ Signature (CA의 서명)
```

4. 공인인증서

HTTPS와 비교

구분	공인인증서	HTTPS (SSL/TLS)
인증 주체	사용자와 웹사이트(또는 기관)	클라이언트(브라우저)와 서버(웹사이트)
인증서 발급자	공인인증기관(KTNET, 금융결제원 등)	인증기관(CA, DigiCert, Let's Encrypt 등)
사용 목적	사용자 신원 확인, 전자서명, 암호화	서버 신원 확인, 통신 암호화
핵심 수단	공개키/개인키, 인증서	공개키/개인키, 서버 인증서

- **HTTPS**는 "웹사이트가 진짜인지"를 보장하는 **서버 신분증**
- **공인인증서**는 "사용자가 진짜 본인인지"를 보장하는 **개인 신분증**

4. 공인인증서

실습

✓ 개인키 서명 / 공개키 검증

```
C:\Users\user\workspaces\coding_assistant> openssl x509 -in certificate(public).pem -text -noout
```

```
Certificate:
```

```
Data:
```

```
Version: 3 (0x2)
```

```
Serial Number:
```

```
4d:b5:3f:89:d0:6f:84:7b:f4:5d:57:2a:f7:ec:04:22
```

```
Signature Algorithm: sha256WithRSAEncryption
```

```
Issuer: O=mkcert development CA, OU=kopo00@Ubuntu-24, CN=mkcert kopo00@Ubuntu-24
```

```
Validity
```

```
Not Before: Oct 25 10:00:24 2025 GMT
```

```
Not After : Jan 25 10:00:24 2028 GMT
```

```
...
```